VOLUME 04 OF 10

Computer Systems and Software Engineering

Intermediate undergraduate

Connect programs to processors, operating systems, concurrency, networks, storage and professional practice.

5 modules / 20 lessons / 60 quiz questions

Prepared September 28, 2026

A guided self-study program with original lesson primers, assigned study, engineering labs, checkpoint projects and research work. Not an accredited degree or a substitute for the required practice and feedback.

Volume map

Module 16 Processor architecture and assembly

16.1 Instruction sets, registers and calling conventions • 16.2 Pipelines, dependencies and branch prediction • 16.3 Caches, locality and memory hierarchy • 16.4 Virtual memory, SIMD and architectural limits

Module 17 Operating systems and resource management

17.1 Processes, threads, system calls and protection • 17.2 Allocation, paging and memory-mapped data • 17.3 Scheduling, responsiveness and real-time constraints • 17.4 Filesystems, persistence and asynchronous I/O

Module 18 Concurrency and parallel programming

18.1 Races, mutexes and happens-before • 18.2 Condition variables, deadlocks and cancellation • 18.3 Atomics, memory ordering and false sharing • 18.4 Task graphs, work stealing and scaling limits

Module 19 Networks, storage and data services

19.1 Network layers, latency and transport semantics • 19.2 Protocol design, serialization and timeouts • 19.3 Relational modeling, SQL and indexes • 19.4 Transactions, consistency and distributed failure

Module 20 Engineering practice, security and maintainability

20.1 Requirements, architecture and design records • 20.2 Defensive programming and threat modeling • 20.3 Build automation, dependencies and release discipline • 20.4 Ethics, accessibility, teamwork and sustainable delivery

Before beginning

Use PROGRAM_GUIDE for sequencing, workload, assessment and the checkpoint project rubric. Use START_HERE for the first session. Reference IDs such as R07 link to assigned resources. The complete reference directory is in PROGRAM_GUIDE and the browser edition.

Answer guidance is separate in ANSWER_KEY.pdf. In the browser edition, answers remain closed until you reveal them. Finish the required study and implementation work rather than treating quiz recall as mastery.

MODULE 16

Processor architecture and assembly

Prerequisites Modules 11-15.

Four lessons

16.1 Instruction sets, registers and calling conventions

Read a small machine-level routine without confusing ISA and implementation.

16.2 Pipelines, dependencies and branch prediction

Explain why instruction count alone does not predict latency.

16.3 Caches, locality and memory hierarchy

Relate access patterns to cache lines and working sets.

16.4 Virtual memory, SIMD and architectural limits

Recognize address translation and vector execution as separate mechanisms.

Module project

Trace a compiled C++ loop through a small assembly example and a cache model. Deliver an instruction/data-flow explanation and a controlled locality experiment.

Core references

R15 MIT OpenCourseWare — 6.004 Computation Structures

R35 RISC-V International — RISC-V ISA specifications

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 16.1 • MODULE 16

Instruction sets, registers and calling conventions

Learning goal Read a small machine-level routine without confusing ISA and implementation.

An instruction-set architecture defines visible operations, registers and relevant behavior. A microarchitecture implements that contract using particular pipelines, caches and execution units. Different implementations can run the same machine code while achieving different performance. A calling convention specifies how functions pass arguments, preserve registers and return values.

Begin with a small load/store ISA such as the selected RISC-V subset, then compare your host's generated assembly. Do not assume an educational subset implements every extension or privileged feature. Assembly understanding helps explain costs and interoperability, but handwritten assembly is rarely the first optimization step.

WORKED EXAMPLE

A load reads a value from memory into a register; an add combines register values; a store writes a result back. A C++ array loop may contain all three, plus loop-control instructions.

PRACTICE + PASS CRITERION

Annotate a ten-to-twenty-instruction routine with register meanings and memory accesses. Pass: predict its output for two inputs and identify caller/callee responsibilities from the chosen ABI reference.

ASSIGNED STUDY

R15: Assembly Language and Procedures and Stacks; R35: selected base integer ISA.

CHECK YOUR UNDERSTANDING

1. How does an ISA differ from a microarchitecture?

2. What does a calling convention specify?

3. Why identify the exact ISA subset in a simulator?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 16.2 • MODULE 16

Pipelines, dependencies and branch prediction

Learning goal Explain why instruction count alone does not predict latency.

A pipeline overlaps stages of multiple instructions. Dependencies can prevent later work from proceeding until earlier results are ready. Modern processors may issue or complete operations out of order while preserving required architectural behavior. Branch prediction speculates about control flow, and mispredictions can discard work.

Throughput measures sustained work per time; latency measures delay for a particular result. A long dependency chain can have poor latency even when the processor can execute many independent operations per cycle. Do not infer performance from clock frequency alone. Instruction mix, memory stalls and available parallelism matter.

WORKED EXAMPLE

Summing one running accumulator creates a dependency chain. Several independent accumulators may expose more instruction-level parallelism, but final combination and numeric-order effects must be considered.

PRACTICE + PASS CRITERION

Compare dependent and independent arithmetic loops while preventing dead-code elimination. Pass: preserve observable results, inspect generated code and explain measured differences as evidence rather than universal CPU facts.

ASSIGNED STUDY

R15: Pipelining and Performance Measures.

CHECK YOUR UNDERSTANDING

1. What is a data dependency?

2. Does higher clock frequency guarantee a faster program?

3. Why can extra independent accumulators help?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 16.3 • MODULE 16

Caches, locality and memory hierarchy

Learning goal Relate access patterns to cache lines and working sets.

Processors use a hierarchy of storage with different capacities and access costs. Caches transfer data in blocks called cache lines. Spatial locality reuses nearby data; temporal locality reuses recently accessed data. A working set is the data actively needed during a phase. Once it exceeds a cache level, behavior can change sharply.

Cache associativity and replacement influence conflicts. A cache miss is not always a main-memory access, because another cache level may satisfy it. Prefetching can help predictable patterns but can also waste bandwidth. Measure several working-set sizes and avoid attributing every slow access to one guessed cache characteristic.

WORKED EXAMPLE

If a hypothetical cache line is 64 bytes and each element is a 4-byte integer, one line can contain 16 consecutive elements when aligned appropriately. A stride of 16 elements can use only one integer from each line.

PRACTICE + PASS CRITERION

Benchmark sequential, strided and randomized access over multiple array sizes. Pass: record assumed versus measured hardware information and explain spatial locality without claiming exact cache behavior from one timing curve.

ASSIGNED STUDY

R15: Caches and the Memory Hierarchy.

CHECK YOUR UNDERSTANDING

1. What is spatial locality?

2. How many 4-byte values fit in a hypothetical 64-byte line?

3. Does every first-level cache miss access DRAM?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 16.4 • MODULE 16

Virtual memory, SIMD and architectural limits

Learning goal Recognize address translation and vector execution as separate mechanisms.

Virtual addresses belong to a process view; address translation maps them to physical storage subject to permissions. A translation lookaside buffer caches recent translations. Large working sets and access patterns can stress translation independently of ordinary data caches.

Single-instruction multiple-data execution applies operations across several lanes. It benefits suitable regular work but needs attention to alignment, tails, masks and numerical semantics. Vector width is not a guaranteed speedup factor. Some workloads are limited by memory bandwidth or serial dependencies instead. Later co-design lessons will connect these observations to roofline and Amdahl-style limits.

WORKED EXAMPLE

Processing four independent additions in vector lanes can reduce instruction overhead, but loading and storing the same bytes may still dominate. The last three elements of a non-multiple-sized array still require correct handling.

PRACTICE + PASS CRITERION

Inspect compiler vectorization reports for a simple loop and compare aligned/misaligned or tail cases safely. Pass: test lengths 0, 1, lane_width-1 and lane_width+1 and explain translation versus caching.

ASSIGNED STUDY

R15: Virtual Memory and Parallel Processing; R11: bounds and arithmetic contracts.

CHECK YOUR UNDERSTANDING

1. What does a TLB cache?

2. Does four-lane SIMD guarantee a fourfold speedup?

3. Why test array lengths not divisible by vector width?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 17

Operating systems and resource management

Prerequisites Module 16.

Four lessons

17.1 Processes, threads, system calls and protection

Explain which resources are isolated and which are shared.

17.2 Allocation, paging and memory-mapped data

Distinguish reserved address space, committed storage and live objects.

17.3 Scheduling, responsiveness and real-time constraints

Distinguish average completion from meeting a deadline.

17.4 Filesystems, persistence and asynchronous I/O

Design data operations that survive partial failure.

Module project

Create a systems lab report covering processes, memory, scheduling and file I/O. Use unprivileged programs and disposable data; do not alter kernel or device settings on a primary machine.

Core references

R16 Remzi and Andrea Arpaci-Dusseau — Operating Systems: Three Easy Pieces

R15 MIT OpenCourseWare — 6.004 Computation Structures

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 17.1 • MODULE 17

Processes, threads, system calls and protection

Learning goal Explain which resources are isolated and which are shared.

A process is an execution environment with an address space and operating-system-managed resources. Threads within a process share memory but have their own execution state. A system call requests a kernel service through a controlled boundary. User/kernel separation protects privileged operations; it does not make application code automatically safe.

Process isolation helps contain faults, while threads allow lower-overhead sharing at the cost of synchronization complexity. Starting a worker process, a thread and an asynchronous task are not interchangeable choices. Identify ownership, failure propagation and communication needs before choosing a mechanism.

WORKED EXAMPLE

Two threads can see the same game-state object, creating race risk. Two isolated processes normally need an explicit communication mechanism to exchange equivalent information.

PRACTICE + PASS CRITERION

Write a program that reports its process identity and runs independent worker tasks. Pass: explain what memory is shared in the chosen approach and how worker failure is reported without privileged operations.

ASSIGNED STUDY

R16: Processes, Process API and Threads.

CHECK YOUR UNDERSTANDING

1. What do threads in one process typically share?

2. What is a system call?

3. Why might processes be preferable to threads for a risky tool?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 17.2 • MODULE 17

Allocation, paging and memory-mapped data

Learning goal Distinguish reserved address space, committed storage and live objects.

An allocator manages storage for application objects; the operating system manages virtual memory and backing resources. Reserving an address range is not the same as having every page physically resident. Page faults can occur when access requires additional mapping or data movement. Application allocation behavior and operating-system paging interact but are not identical.

Memory mapping can present file data through an address range, but it still requires lifetime, bounds and error handling. It does not guarantee zero I/O latency. Avoid assuming that freeing an object immediately reduces the process's visible memory usage: an allocator may retain storage for reuse.

WORKED EXAMPLE

A large mapped file can occupy a large virtual range while only accessed pages become resident. Reading a new region can still incur page faults and storage activity.

PRACTICE + PASS CRITERION

Compare sequential file reads with a read-only mapped-file approach on a small disposable file. Pass: verify identical content and report virtual/resident concepts separately without promising one method is faster.

ASSIGNED STUDY

R16: Address Spaces, Address Translation, Paging and Memory API.

CHECK YOUR UNDERSTANDING

1. Does reserving virtual address space imply all pages are resident?

2. Why may process memory usage stay high after freeing objects?

3. Does memory mapping eliminate storage latency?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 17.3 • MODULE 17

Scheduling, responsiveness and real-time constraints

Learning goal Distinguish average completion from meeting a deadline.

A scheduler chooses runnable work according to its policy. Context switches have costs, and oversubscription can create contention rather than useful parallelism. Throughput-oriented scheduling is not necessarily suitable for a time-critical audio or input path. Real-time reasoning concerns deadlines and worst-case behavior under stated assumptions.

General desktop operating systems do not make arbitrary user programs hard-real-time merely because a thread has high priority. Priority inversion occurs when high-priority work waits indirectly on lower-priority work holding a needed resource. Design short critical sections and bounded work before experimenting with priorities.

WORKED EXAMPLE

An audio callback needed every few milliseconds can glitch if it waits for a worker holding a mutex during a long file load, even when average CPU usage is low.

PRACTICE + PASS CRITERION

Simulate a scheduler or instrument a bounded producer/consumer workload without changing privileged priorities. Pass: show one contention-induced latency spike and propose a structural fix rather than only raising priority.

ASSIGNED STUDY

R16: Scheduling, Multi-Level Feedback Queue and Concurrency Bugs.

CHECK YOUR UNDERSTANDING

1. What is priority inversion?

2. Does low average CPU use guarantee deadlines are met?

3. Why avoid blocking I/O in an audio callback?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 17.4 • MODULE 17

Filesystems, persistence and asynchronous I/O

Learning goal Design data operations that survive partial failure.

A filesystem organizes persistent data and metadata. A successful write call does not always mean data has reached durable storage. Buffering, ordering and crash consistency require explicit treatment when durability matters. A robust save process avoids overwriting the only valid copy with partially generated data.

Asynchronous I/O lets work proceed without blocking the caller, but completion can occur later and fail. Buffers and request objects must remain valid until completion according to the API. Cancellation is a request with a contract, not necessarily immediate erasure of all work. Separate correctness, durability and performance goals.

WORKED EXAMPLE

A common save strategy writes a new temporary file, validates it and performs an appropriate replacement operation. The exact durability guarantee still depends on filesystem/API synchronization and failure assumptions.

PRACTICE + PASS CRITERION

Build a save pipeline with injected failures before, during and after serialization. Pass: recover a valid prior or new save under your documented model and never reuse an asynchronous buffer before completion.

ASSIGNED STUDY

R16: Files and Directories, Crash Consistency and Journaling.

CHECK YOUR UNDERSTANDING

1. Does every successful write imply durable storage?

2. Why retain an asynchronous I/O buffer until completion?

3. What should a save-recovery test inject?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 18

Concurrency and parallel programming

Prerequisites Modules 12 and 16-17.

Four lessons

18.1 Races, mutexes and happens-before

Protect shared invariants using a clear synchronization model.

18.2 Condition variables, deadlocks and cancellation

Make waiting and shutdown correct under adverse timing.

18.3 Atomics, memory ordering and false sharing

Use low-level synchronization only with a complete correctness argument.

18.4 Task graphs, work stealing and scaling limits

Parallelize independent work and measure end-to-end benefit.

Module project

Implement a bounded job system with dependency handling, shutdown, failure propagation and stress tests. Compare one-thread and multi-thread execution on a real workload.

Core references

R11 C++ Core Guidelines editors — C++ Core Guidelines

R16 Remzi and Andrea Arpaci-Dusseau — Operating Systems: Three Easy Pieces

R37 NVIDIA — CUDA Programming Guide

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 18.1 • MODULE 18

Races, mutexes and happens-before

Learning goal Protect shared invariants using a clear synchronization model.

Concurrent execution interleaves operations. A C++ data race occurs when conflicting accesses are not properly ordered and at least one is a write, with the relevant accesses not atomic; this leads to undefined behavior. A race condition is a broader logical timing defect and can exist even when individual accesses are atomic.

A mutex protects a shared invariant only when all relevant participants follow the same locking protocol. The critical section must cover the complete operation that needs to be atomic at the application level. Ownership and message passing can reduce shared mutable state. Document the happens-before relationships rather than relying on "it usually runs first."

WORKED EXAMPLE

Checking balance >= price and then subtracting price in separate unlocked steps can allow two purchases to spend the same funds. Making each field read atomic does not automatically make the whole purchase transaction atomic.

PRACTICE + PASS CRITERION

Implement a shared counter and a compound inventory transaction with proper synchronization. Pass: stress test, inspect with an available race detector and explain why detector silence alone is not proof.

ASSIGNED STUDY

R11: Concurrency and Parallelism; R16: Locks and Concurrency Bugs.

CHECK YOUR UNDERSTANDING

1. Can individually atomic reads/writes still form a logical race?

2. What must a mutex protocol protect?

3. Why is "this thread usually runs first" insufficient?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 18.2 • MODULE 18

Condition variables, deadlocks and cancellation

Learning goal Make waiting and shutdown correct under adverse timing.

A condition variable allows a thread to wait for a predicate to become true while coordinating with a mutex. Wait in a loop because wakeups do not by themselves prove the predicate. Notifications and state changes must follow a consistent protocol to avoid missed progress.

Deadlock can arise from circular resource dependencies. A lock-order rule or reduced lock scope can prevent common cases. Shutdown deserves the same rigor as steady-state execution: wake waiters, stop accepting work, define what happens to queued jobs and join threads safely. Cancellation must specify whether work is abandoned, completed or rolled back.

WORKED EXAMPLE

A consumer waits while queue_empty and not stopping. On shutdown, the producer changes stopping under the agreed synchronization and notifies waiters so they can exit even when no work arrives.

PRACTICE + PASS CRITERION

Create a producer/consumer queue with graceful shutdown and failure propagation. Pass: test stopping with an empty queue, a full queue and active consumers; no thread remains blocked indefinitely.

ASSIGNED STUDY

R16: Condition Variables, Semaphores and Concurrency Bugs; R11: locking guidance.

CHECK YOUR UNDERSTANDING

1. Why test a condition-variable predicate in a loop?

2. What does a consistent lock order prevent?

3. Why must shutdown notify blocked workers?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 18.3 • MODULE 18

Atomics, memory ordering and false sharing

Learning goal Use low-level synchronization only with a complete correctness argument.

Atomic operations provide indivisible access according to their type and memory-order rules. Acquire/release synchronization can publish earlier writes when the specified read observes the release sequence appropriately. Relaxed ordering does not provide the same cross-object publication guarantees. Volatile is not a substitute for C++ thread synchronization.

False sharing occurs when independent frequently written data occupy a shared cache line, causing coherence traffic. Padding may help a measured case but increases footprint. Lock-free algorithms also face reclamation and progress issues; an atomic pointer alone does not make referenced storage safe to free. Prefer a proven library design unless low-level implementation is the explicit learning goal.

WORKED EXAMPLE

A producer writes payload, then publishes a ready flag with release ordering; a consumer that observes the corresponding flag with acquire ordering can safely observe the published payload under the complete protocol.

PRACTICE + PASS CRITERION

Write a publication litmus test with a documented happens-before diagram. Measure a padded versus unpadded counter layout. Pass: do not claim stress tests prove memory-model correctness or assume a fixed cache-line size without labeling it.

ASSIGNED STUDY

R11: atomics and memory model; R16: concurrency; R37: later GPU memory-model comparison.

CHECK YOUR UNDERSTANDING

1. Is volatile a C++ inter-thread synchronization mechanism?

2. What is false sharing?

3. Why is freeing a node difficult in lock-free structures?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 18.4 • MODULE 18

Task graphs, work stealing and scaling limits

Learning goal Parallelize independent work and measure end-to-end benefit.

A task graph expresses jobs and dependencies. Parallel execution is possible among ready tasks that do not conflict. Work stealing distributes queued tasks dynamically, but scheduling overhead can exceed the benefit of tiny jobs. Batch work at a granularity that matches the workload and hardware.

Amdahl's law models the limit from a serial fraction: speedup = 1/(s+(1-s)/p) under its simplifying assumptions. Communication, contention and load imbalance can reduce real speedup further. A job system must handle exceptions, ownership, completion and shutdown, not only start threads. Benchmark the actual simulation rather than an empty task loop.

WORKED EXAMPLE

If 20% of a workload remains serial, an infinite number of ideal workers cannot exceed a 5x speedup. Four ideal workers give 1/(0.2+0.8/4)=2.5x.

PRACTICE + PASS CRITERION

Implement a small task graph for independent entity updates followed by aggregation. Pass: compare 1, 2 and available worker counts, verify results and explain a case where more workers hurt.

ASSIGNED STUDY

R15: Parallel Processing; R16: concurrency; R11: task-oriented concurrency guidance.

CHECK YOUR UNDERSTANDING

1. What is the ideal maximum speedup with a 25% serial fraction?

2. Why can very small tasks reduce performance?

3. What does a task dependency express?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 19

Networks, storage and data services

Prerequisites Modules 15 and 17-18.

Four lessons

19.1 Network layers, latency and transport semantics

Choose transport behavior from application requirements.

19.2 Protocol design, serialization and timeouts

Treat every received byte as untrusted structured input.

19.3 Relational modeling, SQL and indexes

Represent game data with constraints that survive application mistakes.

19.4 Transactions, consistency and distributed failure

Preserve invariants when several operations must succeed together.

Module project

Build a local-only client/server message exercise and a small relational game-data service. Test malformed framing, transaction rollback and duplicate requests without exposing a public service.

Core references

R17 IETF / RFC Editor — RFC 9293: Transmission Control Protocol

R18 PostgreSQL Global Development Group — PostgreSQL tutorial

R16 Remzi and Andrea Arpaci-Dusseau — Operating Systems: Three Easy Pieces

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 19.1 • MODULE 19

Network layers, latency and transport semantics

Learning goal Choose transport behavior from application requirements.

Networks exchange data through layered protocols. IP provides packet delivery without guaranteeing arrival or order. TCP presents a reliable ordered byte stream, not a stream of application messages. UDP preserves datagram boundaries but does not itself supply delivery, ordering or congestion-control behavior. Applications still need explicit protocols.

Latency, jitter, loss and bandwidth describe different constraints. A high-bandwidth link can still have long round-trip delay. Reliable delivery can involve waiting for missing information, which affects real-time design. Use localhost or an authorized lab network for exercises and keep authentication and encryption decisions separate from transport assumptions.

WORKED EXAMPLE

Two send calls over TCP do not guarantee two matching receive calls. The receiver may get part of one message or several messages together and must parse framing correctly.

PRACTICE + PASS CRITERION

Create a localhost client/server exchange with a length-prefixed message. Pass: split one message across several writes and combine several messages in one write; the receiver reconstructs them correctly.

ASSIGNED STUDY

R17: TCP service and interface semantics; R16: networking context.

CHECK YOUR UNDERSTANDING

1. Does TCP preserve application message boundaries?

2. Does high bandwidth imply low latency?

3. What behavior must an application add when using UDP?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 19.2 • MODULE 19

Protocol design, serialization and timeouts

Learning goal Treat every received byte as untrusted structured input.

A protocol defines message types, fields, ranges, versioning and permitted sequences. Serialization must specify byte order and numeric representation. Validate lengths before allocation and cap total resource use. A checksum can detect some corruption but is not equivalent to cryptographic authentication.

Timeouts handle missing progress, not certainty about remote state. A timed-out operation may have executed remotely even if its response was lost. Request IDs and idempotent operations help manage retries. Compatibility is a design decision: unknown fields and versions should be accepted, ignored or rejected according to a documented rule.

WORKED EXAMPLE

A repeated "add 10 coins" request can duplicate a reward. A request with a unique operation ID and server-side deduplication can avoid applying the same accepted operation twice under the defined retention policy.

PRACTICE + PASS CRITERION

Add versioned framing, maximum sizes, timeout handling and request IDs to the lab protocol. Pass: reject oversized and truncated messages before dangerous allocation and test duplicate delivery.

ASSIGNED STUDY

R17: transport failure context; R18: transaction and data-integrity foundations.

CHECK YOUR UNDERSTANDING

1. Does a timeout prove the remote operation never happened?

2. Why validate a length before allocating its buffer?

3. What does idempotence mean?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 19.3 • MODULE 19

Relational modeling, SQL and indexes

Learning goal Represent game data with constraints that survive application mistakes.

A relational schema defines tables, keys and relationships. A primary key identifies a row; a foreign key constrains references between tables. Normalization reduces certain redundancy and update anomalies, while deliberate denormalization trades simpler reads for additional consistency work.

SQL queries express desired results, but their execution plans determine physical work. An index can accelerate suitable lookups while consuming storage and adding write cost. Parameterized queries separate data from executable query structure. Do not build SQL by concatenating untrusted text, and do not assume an index helps every query.

WORKED EXAMPLE

A players table and an inventory table linked by player_id avoid embedding an unbounded list in a single ambiguous text field. A uniqueness constraint on (player_id,item_id) can enforce one aggregate row per item type.

PRACTICE + PASS CRITERION

Create a small schema with primary, foreign and uniqueness constraints. Use parameterized queries and inspect one indexed query plan. Pass: demonstrate rejected invalid references and compare a query before/after indexing.

ASSIGNED STUDY

R18: tables, queries, joins, foreign keys and indexes via the linked manual.

CHECK YOUR UNDERSTANDING

1. What does a foreign key protect?

2. Why can indexes slow writes?

3. Why use parameterized SQL?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 19.4 • MODULE 19

Transactions, consistency and distributed failure

Learning goal Preserve invariants when several operations must succeed together.

A transaction groups related database work under defined atomicity, consistency, isolation and durability guarantees. Isolation levels differ; applications must understand which anomalies remain possible. A transfer should not debit one account without the corresponding credit. Concurrency can require retries even when each statement is individually correct.

Across multiple services, a local transaction does not automatically create a global one. Partial failure, duplicate messages and stale reads require explicit coordination. Separate durable authoritative data from caches. A cache miss is recoverable; losing the only authoritative record is a different failure. Test the system's invariants under interruption rather than only normal response codes.

WORKED EXAMPLE

Transferring five items between players requires both inventories to change together and total quantity to remain constant. A failure between updates must not destroy or create items.

PRACTICE + PASS CRITERION

Implement an inventory transfer with transactions and forced failure between updates. Pass: verify conservation, rollback and duplicate-request handling; document what is not guaranteed across processes or services.

ASSIGNED STUDY

R18: Transactions; R16: persistence and failure reasoning.

CHECK YOUR UNDERSTANDING

1. What does transaction atomicity mean?

2. Does a transaction in one database automatically include another service?

3. What invariant should an item-transfer test preserve?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

MODULE 20

Engineering practice, security and maintainability

Prerequisites Volumes 01-04.

Four lessons

20.1 Requirements, architecture and design records

Make tradeoffs explicit and trace implementation back to needs.

20.2 Defensive programming and threat modeling

Protect assets and trust boundaries before selecting security mechanisms.

20.3 Build automation, dependencies and release discipline

Produce a verifiable artifact from a known source revision.

20.4 Ethics, accessibility, teamwork and sustainable delivery

Evaluate engineering decisions beyond code correctness.

Module project

Complete Gate D: a reviewed systems application with a design record, threat model, CI-style local test pipeline, dependency inventory and an incident/recovery exercise.

Core references

R01 ACM / IEEE-CS / AAAI — CS2023 knowledge areas

R02 ACM / IEEE Computer Society — Computer Engineering Curricula 2016

R05 Scott Chacon and Ben Straub — Pro Git

R11 C++ Core Guidelines editors — C++ Core Guidelines

R29 Microsoft — Xbox Accessibility Guidelines

Study cadence

Treat each lesson as one or more study sessions. Predict and derive before running code, complete the assigned reading and exercises, record lab evidence, then answer the quiz without the key. Review this module’s project before moving on.

LESSON 20.1 • MODULE 20

Requirements, architecture and design records

Learning goal Make tradeoffs explicit and trace implementation back to needs.

A requirement states an observable need or constraint. Functional requirements describe behavior; quality requirements describe properties such as latency, reliability or accessibility. Architecture allocates responsibilities and dependencies to meet those requirements. A diagram without contracts and failure behavior is not enough.

An architecture decision record states the problem, constraints, alternatives, chosen approach and consequences. Avoid premature distributed systems or custom frameworks when a smaller design meets the need. Revisit decisions when evidence changes. Traceability links requirements to tests and avoids declaring success merely because all planned code was written.

WORKED EXAMPLE

"Loads quickly" is vague. "Loads the defined test level within a chosen threshold on the declared baseline machine, measured from action to usable state" is testable once the threshold and procedure are filled in.

PRACTICE + PASS CRITERION

Write six measurable requirements and two decision records for the simulation. Pass: every requirement has an acceptance test and each decision includes a rejected alternative and a cost.

ASSIGNED STUDY

R01: Software Engineering; R02: engineering design and professional practice.

CHECK YOUR UNDERSTANDING

1. What makes a requirement testable?

2. Why record rejected alternatives?

3. Is completing planned tasks the same as satisfying requirements?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 20.2 • MODULE 20

Defensive programming and threat modeling

Learning goal Protect assets and trust boundaries before selecting security mechanisms.

A threat model identifies assets, actors, entry points, trust boundaries and plausible failures. Game clients, mods, save files and asset importers are not automatically trusted. Validate input, minimize privileges and isolate risky processing. Memory-safe interfaces, bounds checks, fuzzing and sanitizers help address different defect classes.

Authentication establishes an identity or credential; authorization decides permitted actions. Encryption alone does not enforce game rules or protect a compromised endpoint. Use established security libraries rather than inventing cryptography. Perform testing only against your own code or authorized lab targets; the purpose here is defensive engineering.

WORKED EXAMPLE

A client may request a move, but an authoritative server should validate speed and permissions rather than accepting the client's claimed final position as truth.

PRACTICE + PASS CRITERION

Threat-model the save parser and local service. Add bounds checks and a small malformed-input fuzz harness. Pass: cap work/memory, preserve useful errors and record residual risks instead of claiming complete security.

ASSIGNED STUDY

R01: Security; R11: bounds/lifetime safety; R02: information-security breadth.

CHECK YOUR UNDERSTANDING

1. How does authorization differ from authentication?

2. Why is a game client a trust boundary?

3. Does encryption replace input validation?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 20.3 • MODULE 20

Build automation, dependencies and release discipline

Learning goal Produce a verifiable artifact from a known source revision.

Continuous integration runs repeatable checks when changes are made. A useful pipeline builds, tests and reports failures from a clean environment. Pin dependencies sufficiently to reproduce the result and record toolchain versions. A dependency inventory includes origin, version and license obligations; it is not enough to know that a package downloaded successfully.

Separate secrets from source and limit automation permissions. Preserve symbols or diagnostics needed to investigate crashes, while controlling access to sensitive information. A release artifact should be traceable to a commit and build configuration. Automated success is evidence only for the checks actually run.

WORKED EXAMPLE

A release manifest can record source commit, compiler, build type, dependency versions and asset bundle hash. This helps determine whether a reported defect came from the same artifact that was tested.

PRACTICE + PASS CRITERION

Create a local clean-build script that runs tests and produces a versioned archive plus manifest. Pass: run it from a fresh build directory and deliberately break a test to confirm the pipeline stops.

ASSIGNED STUDY

R05: workflows and tagging; R12: builds and testing; R01: software engineering.

CHECK YOUR UNDERSTANDING

1. What is the purpose of dependency pinning?

2. What should identify a release artifact's origin?

3. Why test a pipeline with an intentional failure?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.

LESSON 20.4 • MODULE 20

Ethics, accessibility, teamwork and sustainable delivery

Learning goal Evaluate engineering decisions beyond code correctness.

Engineering work affects players, colleagues and the public. Accessibility, privacy, intellectual property, honest reporting and sustainable maintenance belong in technical decisions. Obtain permission for assets and code and respect the licenses under which they are supplied. Do not assume publicly visible material is free to redistribute.

Teams need clear responsibilities, respectful review and realistic scope. Crunch is not a substitute for planning and can hide systemic delivery problems. A postmortem should identify contributing conditions and improvements, not scapegoats. For hardware, safety and environmental constraints become additional design requirements, not optional polish.

WORKED EXAMPLE

A performance optimization that removes readable subtitles or makes controls non-remappable may violate the product's requirements despite increasing frame rate.

PRACTICE + PASS CRITERION

Run an accessibility review and a blameless failure retrospective on the project. Pass: identify at least three actionable changes, assign acceptance evidence and document third-party attribution honestly.

ASSIGNED STUDY

R29: input, text, captions and UI guidance; R01/R02: professional and ethical practice.

CHECK YOUR UNDERSTANDING

1. Does public availability imply permission to redistribute?

2. Why include accessibility in acceptance criteria?

3. What should a blameless retrospective seek?

Attempt all three questions before opening the answer key. A lesson is complete only after the assigned study and lab also pass.